Codificar el software asegurando la calidad de acuerdo con estándares de la industria

Desarrollar la solución de software de acuerdo con el diseño y metodológicas de desarrollo

Instructora:
Yazmin Martinez Bejarano

Aprendiz:
Danna Franchesca Perez Gonzalez

23/10/2026

PREGUNTAS DE INICIO

1. ¿Qué diferencia hay entre un selector de clase y uno de id, y cuál conviene para elementos que se repiten?

- : Un selector de clase se escribe con un punto (`.`) y se puede utilizar en varios elementos. Un selector de ID se escribe con # y debe identificar un elemento único en la página. Para elementos que se repiten, conviene utilizar una clase.

2. ¿Qué cambia box-sizing: border-box en el cálculo del ancho de una caja?

- Hace que el ancho declarado incluya el contenido, el relleno (`padding`) y los bordes (`border`), evitando que la caja sea más ancha de lo esperado.

3. ¿Dónde se declara una variable como --color-primario y cómo se usa después?

- Se declara normalmente dentro de `:root` para que esté disponible en toda la página y se utiliza con la función `var()`.

4. Si dos reglas con la misma especificidad afectan a la misma propiedad, ¿cuál gana? (Hoy esta respuesta será clave).

- Gana la regla que aparece más abajo en la hoja de estilos, porque cuando ambas tienen la misma especificidad, se aplica la última regla declarada.

Pregunta de participación: si un <a> es inline, ¿por qué en el menú de Café Aroma tiene display: block? (Pista: observa su relleno).

- Porque `display: block` permite que el enlace ocupe toda la línea disponible y que su relleno (`padding`) se aplique cómodamente, creando un área más grande y fácil de pulsar.

Pregunta de participación: si tienes tres cajas en un contenedor flex y quieres la primera pegada a la izquierda y las otras dos a la derecha, ¿qué propiedad probarías primero?

- `margin-left: auto` en la segunda caja. Esto empuja la segunda y las siguientes hacia la derecha

Pregunta de participación: en repeat(3, 1fr), ¿qué significa el 3 y qué significa 1fr? ¿Qué pasaría con repeat(4, 1fr)?

- El 3 indica que se crean tres columnas y `1fr` significa que cada columna recibe una fracción igual del espacio disponible. Con `repeat(4, 1fr)` se crearían cuatro columnas del mismo tamaño.

Pregunta de participación: una pantalla de 700 px, ¿cae en la vista de tableta o en la de celular según nuestra tabla? ¿Y una de 600 px exactos?

- 700 px: tableta, porque supera los 600 px.
- 600 px exactos: celular, porque entra en la condición `max-width: 600px`.

Preguntas de análisis

Responde por escrito. En las preguntas de código, escribe la corrección en tu cuaderno o en un archivo.

1. ¿Qué diferencia práctica hay entre display: block y display: inline-block?

A) block ocupa toda la línea y empieza en una línea nueva; inline-block se coloca junto a otros elementos, pero acepta ancho y alto.

B) block permite ancho y alto; inline-block los ignora y se comporta como si fuera texto.

C) inline-block ocupa toda la línea y block se coloca junto a otros elementos.

D) No hay diferencia visible; solo cambia el nombre de la propiedad.

2. Un contenedor flex tiene tres cajas con ancho propio. Solo a la segunda se le aplica flex:

1. ¿Qué ocurre?

A) Las tres cajas quedan exactamente del mismo ancho.

B) La segunda se estira y ocupa el espacio libre que dejan las otras dos.

C) La segunda pasa a una línea nueva debajo de las otras.

D) La segunda desaparece porque flex: 1 no es un valor válido.

3. Con grid-template-columns: 200px 1fr 2fr, ¿cómo se reparte el ancho del contenedor?

A) Las tres columnas miden 200 px.

B) La primera mide 200 px, la segunda 1 px y la tercera 2 px.

C) La primera mide 200 px; el espacio restante se divide en tres partes: una para la segunda
columna y dos para la tercera.

D) La primera mide 200 px y las otras dos se reparten el ancho por igual.

4. Una página tiene @media (max-width: 900px) y, a continuación, @media (max-width: 600px). Se abre en una ventana de 750 px de ancho. ¿Qué bloques se aplican?

A) Solo el de 600 px.

B) Ambos bloques.

C) Ninguno, porque 750 px no es ni 900 ni 600.

D) Solo el de 900 px.

5. Para que un enlace o botón sea cómodo de tocar en un celular, ¿qué tamaño mínimo se recomienda?

A) Unos 20 × 20 px, porque el dedo es pequeño.

B) Alrededor de 44 × 44 px, incluyendo su relleno.

C) El tamaño no importa si el texto tiene buen contraste.

D) Al menos 200 × 200 px para que nunca se falle.

6. Basta con escribir <meta name="viewport" content="width=device-width, initial-scale=1.0"> para que una página con anchos fijos de 1100 px se adapte sola a un celular.

( ) Verdadero **( ) Falso Justifica**: La etiqueta `meta viewport` configura la escala y el ancho de la página en dispositivos móviles, pero no hace que los elementos con anchos fijos se adapten automáticamente. Se necesitan estilos responsive, como porcentajes, Grid, Flexbox y media queries.

7. Un elemento con display: none desaparece de la página y deja de ocupar espacio.

( ) Verdadero ( ) Falso Justifica:El elemento desaparece visualmente y deja de ocupar espacio en el diseño de la página.

8. Con gap: 1rem en un contenedor flex o grid, la separación se aplica solo entre los elementos, no antes del primero ni después del último.
(**) Verdadero** ( ) Falso **Justifica**:`gap` crea separación entre los elementos de Flexbox o Grid, pero no añade espacio antes del primero ni después del último.

9. justify-content siempre alinea en horizontal y align-items siempre en vertical, sin importar el valor de flex-direction.
( ) Verdadero ( ) **Falso Justifica**:En Flexbox, depende de `flex-direction`. Con `row`, el eje principal es horizontal; con `column`, es vertical. `justify-content` actúa sobre el eje principal y `align-items` sobre el eje transversal.

10. Tu página tiene media queries, pero en un celular real se ve como un escritorio reducido y las media queries parecen no funcionar. ¿Qué línea del <head> probablemente falta y por qué importa?
Respuesta: <meta name="viewport" content="width=device-width, initial-scale=1.0"> Esta línea permite que el ancho del diseño se ajuste al ancho real del dispositivo y que las media queries responsive funcionen según ese ancho.

11. Explica con tus palabras cuándo conviene Flexbox y cuándo Grid. Da un ejemplo de cada uno tomado de la página de Café Aroma.
Respuesta:Flexbox conviene para organizar elementos en una sola dirección, en fila o columna. En Café Aroma se puede usar para alinear el encabezado y el menú.

Grid conviene para organizar elementos en filas y columnas. En Café Aroma se usa en las tarjetas de productos para distribuirlas en varias columnas.

12. Una imagen de 1600 px de ancho se corta y provoca desplazamiento horizontal en un celular. ¿Qué dos declaraciones aplicarías a todas las imágenes, qué hace cada una y para qué sirve object-fit: cover?
Respuesta:`max-width: 100%` impide que la imagen sea más ancha que su contenedor.
`height: auto` conserva sus proporciones y evita deformarla.
`object-fit: cover` permite que la imagen llene el espacio asignado recortando la parte que sobra, en lugar de deformarse. Es útil cuando se busca que las imágenes tengan un tamaño uniforme.

13. Describe la diferencia entre diseñar «escritorio primero» y «móvil primero». ¿Cuál se usó en el taller, qué condición de media query emplea cada enfoque y qué ventaja tiene el segundo?

Respuesta:En el enfoque escritorio primero, se empieza diseñando para pantallas grandes y se ajusta para pantallas pequeñas, normalmente con media queries `max-width`.

En el enfoque móvil primero, se empieza con el diseño para celulares y se agregan mejoras para pantallas mayores mediante `min-width`.

En el taller inicialmente usamos escritorio primero y luego practicamos móvil primero. La ventaja de móvil primero es que prioriza los dispositivos pequeños y permite construir un diseño responsive de forma progresiva.

14. Los tres enlaces del menú siguen apareciendo uno debajo del otro. Explica por qué y corrige el CSS.
index.html y styles.css

```
<nav class="menu-principal">
<ul>
<li><a href="#inicio">Inicio</a></li>
<li><a href="#menu">Carta</a></li>
<li><a href="#contacto">Contacto</a></li>
</ul>
</nav>
```

```
/* styles.css */
.menu-principal {
display: flex;
gap: 1rem;
}
.menu-principal ul {
list-style: none;
}
```

Respuesta: Porque `display: flex` está aplicado a `.menu-principal`, que contiene el `<ul>`, pero no a los elementos `<li>` que contienen los enlaces. Por eso, los elementos de la lista siguen organizándose verticalmente.

15. Las tarjetas aparecen apiladas en escritorio y el diseño nunca cambia en el celular. Encuentra los dos errores y corrígelos.
styles.css

```
.tarjetas {
display: grid;
grid-template-column: repeat(3, 1fr);
gap: 1.5rem;
}
@media (max-width: 600) {
.tarjetas {
grid-template-columns: 1fr;
}
```

}
Respuesta: `grid-template-column` debe ser `grid-template-columns`.
A `max-width: 600` le falta la unidad `px`.

.tarjetas { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1.5rem;}

@media (max-width: 600px) { .tarjetas { grid-template-columns: 1fr; }}

16. En un celular de 400 px de ancho, ¿cuántas columnas de tarjetas se ven con este CSS? Justifica y reordénalo para que se vea una sola.
styles.css
.tarjetas {
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 1.5rem;
}
@media (max-width: 600px) {
.tarjetas { grid-template-columns: 1fr; }
}
@media (max-width: 900px) {
.tarjetas { grid-template-columns: repeat(2, 1fr); }
}
Respuesta: Aparecen dos columnas, porque ambas media queries se aplican a 400 px. La regla de `max-width: 900px` aparece después y sobrescribe la regla de `max-width: 600px` porque ambas tienen la misma especificidad.